



Universidad Autónoma de Aguascalientes

Centro de Ciencias Básicas

Departamento de Sistemas Electrónicos

Ing. En Sistemas Computacionales

Semestre: 8. Grupo: "C"

Materia: Seguridad en Sistemas

Profesor: Arturo Ocampo Silva

Alumna: Ilse Jacqueline Martínez Espinosa

ID: 349964

"Actividad 1: Primeros Pasos Cifrando"

Fecha entrega: 10/02/2026

Aguascalientes, Ags.

Contenido

Introducción.....	3
Objetivo	3
Desarrollo	4
Gestión Dinámica del Alfabeto	4
Filtración del Texto de entrada.....	4
Métodos de cifrado	5
Interacción y Gestión de Resultados.....	6
Conclusión.....	6
Bibliografías	6

Introducción

A lo largo de la historia, la necesidad de secreto ha sido fundamental. Tanto los gobiernos como la gente común han buscado cada vez más asegurar la entrega de ciertos mensajes e información importante de forma que solo el destinatario previsto pueda acceder a ellos y comprenderlos. Esta necesidad de secreto impulsó la invención y el arte de la ocultación, la codificación y la creación de códigos. A cambio, la necesidad de inteligencia e información condujo al desarrollo de técnicas de descifrado de códigos. Estas técnicas atacan principalmente una debilidad específica que pueda tener un código o método de ocultación, haciendo que la información buscada sea evidente y comprensible para el atacante.

Abu Yusuf Ya'qub ibn Ishaq Al-Kindi, (أبو يوسف يعقوب بن إسحاق الكندي) mejor conocido en Occidente como Alkindous, era un árabe étnico descendiente de la tribu real Kindah.

Al-Kindi describió el criptoanálisis en dos frases de su tratado más importante, titulado "Manuscrito sobre el descifrado de mensajes criptográficos", que dice:

"Una forma de resolver un mensaje cifrado, si conocemos su idioma, es encontrar un texto plano diferente del mismo idioma, lo suficientemente largo como para llenar una hoja aproximadamente, y luego contar las ocurrencias de cada letra. Llamamos a la letra más frecuente 'primera', a la siguiente letra más frecuente 'segunda', y así sucesivamente [...]. Luego examinamos el texto cifrado que queremos resolver y clasificamos sus símbolos. Encontramos el símbolo más frecuente y lo cambiamos a la forma de la 'primera' letra [...], hasta que contemos todos los símbolos del criptograma".

La técnica de Al-Kindi llegó a conocerse como análisis de frecuencia, el cual consiste en calcular los porcentajes de aparición de las letras en un idioma y compararlos con los símbolos del texto cifrado. Este desarrollo fue un punto de inflexión en el hackeo de sistemas de encriptación simples, ya que demostró matemáticamente que los cifrados de sustitución mono-alfabética conservan la huella estadística del idioma original.

Es precisamente por este principio que métodos clásicos como el cifrado César y el Atbash ya no son viables como métodos de protección de datos en la actualidad. Ambos son sistemas de sustitución simple; el César simplemente desplaza las letras un número fijo de posiciones, mientras que el Atbash invierte el alfabeto. Al no alterar la frecuencia natural de las letras, cualquier mensaje encriptado con ellos puede ser descifrado fácilmente usando el método de Al-Kindi. Además, su espacio de claves es tan reducido (25 posibles desplazamientos para César y solo 1 regla fija para Atbash) que, en la era de la computación moderna, cualquier sistema informático puede romperlos mediante fuerza bruta en fracciones de segundo, volviéndolos completamente inútiles para la seguridad de la información moderna. [1]

Objetivo

Desarrollar una aplicación web capaz de cifrar y descifrar mensajes de texto utilizando los algoritmos clásicos de criptografía César y Atbash, tomando como base el código ASCII. El sistema permitirá al usuario ingresar una oración, definir el conjunto de caracteres a emplear

y seleccionar el método de cifrado o descifrado deseado, así como el módulo correspondiente en el caso del cifrado César. Además, la aplicación deberá identificar y aplicar correctamente el tipo de cifrado seleccionado, garantizando el funcionamiento adecuado del proceso y mostrando el resultado en tiempo real a través de una interfaz accesible en la web.

Desarrollo

En general el sistema es una aplicación web interactiva diseñada para cifrar y descifrar mensajes utilizando los algoritmos clásicos de César y Atbash.

El usuario selecciona el conjunto de caracteres (abecedario) que limitará el cifrado. Luego, elige el módulo criptográfico (César o Atbash), la acción a realizar (Cifrar o Descifrar) y, si aplica, el número de posiciones a desplazar, para enseguida escribir el mensaje en el área de texto y gracias a la reactividad la conversión se hace al instante. También cuenta con un portapapeles y un botón de invertir proceso para comprobar visualmente que el mensaje puede regresar a su estado original.

Ahora describiré las funciones principales de la lógica de este sistema.

Nota: Las referencias al código son al enlace de github específicamente en esta dirección de carpeta `src/app/cifrador/cifrador.ts`

Gestión Dinámica del Alfabeto

Se creó un diccionario llamado `presets` el cual almacena diferentes colecciones de caracteres como su nombre lo dice “predefinidos” para que el usuario pueda definir el conjunto de caracteres a emplear, siendo la primera opción la que contiene todos los caracteres del alfabeto que utilizamos y de ahí las demás opciones son subramas de caracteres del primer alfabeto separados por categorías. [Referencia en código \[A\]](#).

A través de la función `onPresetChange()`, el sistema enlaza el menú desplegable de la vista con la lógica interna, siendo esta: cuando el usuario elige una opción en la pantalla (por ejemplo, 'Solo Minúsculas'), esta función busca ese texto en nuestro diccionario y actualiza inmediatamente la variable principal `alphabet`.

Esta función es importante ya que la lógica de las fórmulas de César y Atbash dependen de la cantidad total de caracteres, así que al actualizar la variable `alphabet`, la matemática del sistema detecta el nuevo tamaño y ajusta sus cálculos automáticamente. Así, el cifrado funciona a la perfección sin importar el tamaño del abecedario que el usuario eligió. [Referencia en código \[A1\]](#).

Filtración del Texto de entrada

Se creó una función `limpiarTexto()` la es prácticamente un sistema de validación para que el usuario solo pueda ingresar los caracteres que están presentes en alfabeto que eligió.

Cada vez que el usuario teclea algo, la función entra en acción y revisa el texto letra por letra usando un ciclo for. Con la ayuda del método `.includes()`, verifica si cada carácter existe dentro de nuestro abecedario permitido.

Si la letra es válida, la guarda en una nueva cadena de texto. Si el usuario tecleó un símbolo inválido (por ejemplo, un número cuando eligió 'Solo letras'), simplemente lo ignora y lo deja fuera.

Finalmente, la función actualiza la caja de texto (`this.inputText = textoValido`), reemplazando lo que escribió el usuario únicamente con los caracteres que sí pasaron la validación. De esta forma, los caracteres que no entren en esa opción desaparecen de la pantalla al instante."

[Referencia en código \[B\]](#).

Métodos de cifrado

Para aplicar la criptografía, el sistema utiliza una variable llamada `metodoCifrado`. Esta funciona como un interruptor que le indica al programa qué algoritmo debe ejecutar: el Cifrado César o el Cifrado Atbash. En la interfaz gráfica, esta variable está conectada directamente al menú donde el usuario elige el método de su preferencia. *[Referencia en código \[C\]](#).*

La lógica principal del programa se encuentra en la función `outputText()`. A diferencia de otras funciones, esta es un método reactivo ("getter"), lo que significa que se ejecuta automáticamente en tiempo real cada vez que el usuario escribe una letra, cambia el abecedario o modifica el número de posiciones.

Su lógica interna funciona de la siguiente manera:

Primero, el algoritmo calcula el tamaño exacto del abecedario actual y lo guarda en la variable `n`. Después, toma el texto de entrada y lo analiza letra por letra. Con la ayuda del método `.indexOf()`, el sistema busca en qué posición (índice) se encuentra cada letra dentro de nuestro abecedario. Si por alguna razón la letra no se encuentra (por ejemplo, un salto de línea), el programa simplemente la deja intacta en el resultado para no romper el mensaje.

Una vez que encuentra la posición de la letra, el programa revisa qué método eligió el usuario en `metodoCifrado`:

Si el método es César: El programa toma la posición original de la letra y le suma (para cifrar) o le resta (para descifrar) el número de posiciones que el usuario configuró en la variable `shift`. Para evitar que el sistema marque un error al llegar al final del abecedario, utiliza una operación matemática llamada "módulo" (`% n`). Además, tiene un seguro para cuando se descifra: si la resta da un número negativo, le suma el tamaño del abecedario para dar la vuelta hacia atrás de forma segura.

Si el método es Atbash: El programa ignora la cantidad de posiciones y aplica un reflejo. Para lograrlo, resta la posición actual de la letra al tamaño total del abecedario menos uno (`n - 1`) - `index`. Esto invierte la letra (la primera se cambia por la última, la segunda por la penúltima, etc.) adaptándose a cualquier tamaño de abecedario que el usuario haya seleccionado.

Finalmente, el programa junta todas estas nuevas letras calculadas y las envía directamente a la caja de texto de "Resultado" en la pantalla de forma instantánea. *Referencia en código [C1]*.

Para ayudarme a comprender la función de estos 2 métodos consulte la referencia [2] y [3].

Interacción y Gestión de Resultados

Para mejorar la experiencia del usuario y facilitar las pruebas del sistema, se implementaron dos funciones finales en la interfaz gráfica.

Por un lado, la función `copiarAlPortapapeles()` permite extraer el texto ya procesado de una manera rápida. Al hacer clic en el botón, el sistema utiliza una herramienta interna del navegador para guardar el resultado en la papelera de copiado de la computadora. Además, para darle una confirmación visual al usuario de que la acción funcionó, el sistema cambia temporalmente el color y el texto del botón a "¡Copiado!" durante dos segundos antes de regresar a la normalidad. *Referencia en código [D]*.

Por otro lado, se creó la función `invertirProceso()`, la cual actúa como un comprobador instantáneo. Su trabajo principal es tomar el texto que acaba de salir en la caja de "Resultado" y mandarlo automáticamente de regreso a la caja de "Texto de Entrada". Al mismo tiempo, cambia la acción seleccionada a su opuesto (por ejemplo, si estaba en "Cifrar", lo cambia a "Descifrar").

Esto implementado como opción si uno quiere descifrar lo que acabamos de cifrar y viceversa, como el sistema trabaja en tiempo real, esto hace que el texto se recalcule y regrese inmediatamente a su mensaje original. Esto sirve como una demostración rápida y visual de que las matemáticas del cifrado son reversibles y funcionan a la perfección en ambas direcciones. Además, dejando la opción de poder seleccionar en las opciones el descifrado para que el usuario pueda introducir un texto que previamente él cifro manualmente y poder descifrarlo. *Referencia en código [D1]*.

Conclusión

Gracias a la realización de esta actividad pude comprender el funcionamiento de la criptación por medio de atbash y cesar, al inicio tenía miedo ya que pensaba que era una actividad difícil pero mientras iba investigando me di cuenta de lo sencilla que era y eso me llevó al por que ya no es factible usarlas hoy en día por que son muy fácil de descifrar haciéndolas inseguras.

Bibliografías

[1] Nizamoglu, C. (2003, junio 9). Al-kindí, cryptography, code breaking and ciphers. Muslim Heritage. <https://muslimheritage.com/al-kindí-cryptography/>

[2] Atbash. (s. f.). <https://rumkin.com/tools/cipher/atbash/>

[3] Maths, S. (s. f.). El método César. <https://www.sangakoo.com/es/temas/el-metodo-cesar>